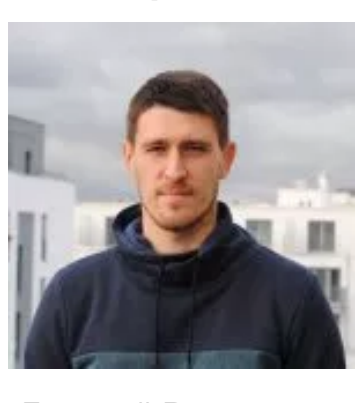
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Kotlin
Корутины
- suspend function
- scope
- builders
- exceptions
- flow

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 11. Scope

21 июня 2021

В этом уроке разбираемся, как работает Scope.

Тема scope - очень непростая для понимания и объяснения. Вполне возможно, что с первого раза не все будет понятно. Рекомендую почитать что-нибудь еще или перечитать этот урок несколько раз.

В прошлом уроке мы разобрали время жизни компонента. Теперь давайте посмотрим какое отношение к этому имеет scope.

Будем продолжать пример с компонентом AppComponent и его сабкомпонентом OrderComponent, который используется в OrderActivity. Напомню, что OrderActivity у нас открывается и закрывается несколько раз за время работы приложения.

Добавим объект OrderRepository:

```
class OrderRepository @Inject constructor() {
    // ...
}
```

Обратите внимание. У него @Inject конструктор. Напомню, это означает, что компонентам не нужны модули, чтобы создать этот объект. Компоненты просто используют этот @Inject конструктор напрямую.

О том, как scope работает в случае Provides методов, поговорим в конце этого урока.

Объект OrderRepository нужен нам в OrderActivity. Получать его будем от сабкомпонента OrderComponent. Добавим в него метод:

```
@Subcomponent
interface OrderComponent {
    fun getOrderRepository(): OrderRepository
}
```

Теперь мы в OrderActivity можем от сабкомпонента OrderComponent получить OrderRepository:

```
val orderComponent = (application as App).appComponent.getOrderComponent()
val orderRepository = orderComponent.getOrderRepository()
```

Тут все знакомо и понятно.

Синглтон

Но что если этот репозиторий нужен нам не только в самом OrderActivity, но и в каком-то из его фрагментов. Чтобы получить репозиторий, фрагмент возьмет у Activity сабкомпонент и сам вызовет getOrderRepository метод.

В итоге сабкомпонент создаст репозиторий и для OrderActivity и для фрагмента. И это будут два разных экземпляра репозитория. Потому что компонент (или сабкомпонент) создает нам новый объект каждый раз когда мы у него этот объект просим. Первым просителем было Activity, вторым - фрагмент.

В итоге мы имеем два экземпляра репозитория OrderRepository, и это нам не очень подходит. Потому что, обычно, такой объект, как репозиторий, мы держим в единственном экземпляре (Singleton). И этот экземпляр используется различными частями приложения.

Нам надо сделать так, чтобы сабкомпонент в методе getOrderRepository всегда возвращал нам один и тот же экземпляр OrderRepository.

Даггер дает нам такую возможность. Мы можем сказать компоненту (или сабкомпоненту), чтобы он создавал нужный нам объект только один раз, хранил его у себя, и возвращал его нам при каждом вызове get метода. Т.е. мы можем от компонента получать синглтон объекта.

scope аннотация

Чтобы научить компонент предоставлять нам синглтоны объектов, необходимо создать для него scope аннотацию. В нашем примере для сабкомпонента OrderComponent мы создадим scope аннотацию OrderScope:

```
@Scope
@Retention(AnnotationRetention.RUNTIME)
annotation class OrderScope
```

Имя аннотации должно описывать время жизни компонента. Чуть позже станет понятно, зачем.

Добавляем аннотацию к интерфейсу сабкомпонента:

```
@OrderScope
@Subcomponent(modules = [OrderModule::class])
interface OrderComponent
```

Теперь он готов создавать синглтоны, но пока что не будет этого делать. Не может же он просто включить синглтоны для всех объектов, которые он создает. Нам нужно будет указать ему, какие именно объекты мы хотим видеть синглтонами. А пока мы этого не сделали, сабкомпонент продолжает работать в обычном режиме, создавая новый объект при каждом вызове get метода.

Чтобы убедиться в этом, можно заглянуть в исходники метода getOrderRepository. Там будет такой код:

```
public OrderRepository getOrderRepository() {
      return new OrderRepository();
    }
```

Видно, что при каждом вызове создается новый объект

Кажется, что с добавлением scope аннотации ничего не поменялось. Но на самом деле это не совсем так. Когда мы добавляем компоненту (или сабкомпоненту) scope аннотацию, это приводит к изменениям в логике его работы. Теперь, при создании какого-либо объекта, компонент проверяет - а не помечен ли этот объект той же scope аннотацией, что и сам компонент. Если не помечен, то объект создается в обычном режиме. Если помечен - то создается синглтон.

Если перенести это на наш пример, получится следующее.

К сабкомпоненту OrderComponent мы добавили scope аннотацию OrderScope. Тем самым мы в сабкомпоненте включили проверку на наличие этой аннотации у создаваемых объектов.

Мы просим OrderComponent создать нам OrderRepository. Сабкомпонент проверяет объект OrderRepository, видит, что объект не помечен аннотацией OrderScope, и создает нам этот объект в обычном (не синглтон) режиме. Если мы хотим получать синглтон OrderRepository объекта, к нему надо добавить аннотацию OrderScope

Давайте сделаем это:

```
@OrderScope
class OrderRepository @Inject constructor()
```

Теперь сабкомпонент OrderComponent при создании этого объекта увидит на нем свою же scope аннотацию и создаст нам синглтон объекта OrderRepository.

Посмотрим, как теперь выглядят исходники метода OrderComponent.getOrderRepository:

```
public OrderRepository getOrderRepository() {
  return orderRepositoryProvider.get();
}
```

Видно, что объект OrderRepository не создается при каждом вызове метода, а запрашивается у провайдера. Именно этот провайдер и отвечает за создание и хранение синглтона в сабкомпоненте.

Чтобы убедиться, что мы действительно получаем синглтон, давайте вызовем метод getOrderRepository в OrderActivity два раза подряд:

```
val orderComponent = (application as App).appComponent.getOrderComponent()

val orderRepository1 = orderComponent.getOrderRepository()
val orderRepository2 = orderComponent.getOrderRepository()

Log.d(TAG, "orderRepository1  = $orderRepository1")
Log.d(TAG, "orderRepository2  = $orderRepository2")
```

Результат:

```
orderRepository1 = ru.startandroid.daggercourse.order.OrderRepository@1ab6fd0
orderRepository2 = ru.startandroid.daggercourse.order.OrderRepository@1ab6fd0
```

Оба раза получаем один и тот же объект. Синглтон работает.

Итак. Чтобы получать от компонента (или сабкомпонента) синглтон объекта, надо:
1) Для компонента создать scope аннотацию и повесить ее на компонент. Тем самым мы у компонента включаем режим создания синглтонов.
2) Повесить эту же scope аннотацию на объект, который хотим видеть синглтоном. Для компонента это будет сигналом, что именно этот объект надо создавать как синглтон. Объекты без этой аннотации будут создаваться компонентом в обычном (не синглтон) режиме.

Время жизни синглтона

Получается, что пока мы находимся на экране OrderActivity, сабкомпонент OrderComponent возвращает нам один и тот же экземпляр репозитория при каждом вызове метода getOrderRepository. Теперь и OrderActivity и все его фрагменты получат один и тот же экземпляр репозитория.

Экземпляр этот хранится в сабкомпоненте OrderComponent. Когда мы закроем OrderActivity, уничтожится сабкомпонент OrderComponent, а с ним и синглтон OrderRepository.

Когда мы в следующий раз откроем этот экран, от AppComponent мы получим новый экземпляр сабкомпонента OrderComponent. Этот сабкомпонент создаст нам новый синглтон OrderRepository.

Давайте проверим это. Откроем OrderActivity и вызовем метод getOrderRepository два раза подряд, как мы это делали чуть раньше.

Результат:

```
orderRepository1 = ru.startandroid.daggercourse.order.OrderRepository@ff8e47d
orderRepository2 = ru.startandroid.daggercourse.order.OrderRepository@ff8e47d
```

Это у нас один синглтон.

Теперь закроем OrderActivity, откроем его снова и вызовем getOrderRepository два раза. Результат:

```
orderRepository1 = ru.startandroid.daggercourse.order.OrderRepository@a711ef7
orderRepository2 = ru.startandroid.daggercourse.order.OrderRepository@a711ef7
```

Снова синглтон, но уже другой, не тот же самый, что был во время прошлого открытия экрана.

Если вы привыкли, что синглтон - это что-то такое, что живет все время работы приложения, то отвыкайте. Даггер создает для нас синглтон, который живет, пока жив компонент, создавший этот синглтон. А компонент запросто может жить меньше, чем приложение.

В нашем примере синглтон OrderRepository ограничен временем жизни сабкомпонента OrderComponent, а тот в свою очередь ограничен экраном OrderActivity.

А что если мы хотим увеличить время жизни этого синглтона. Т.е. чтобы при каждом открытии экрана OrderActivity у нас не создавался бы новый синглтон, а всегда использовался один и тот же. Т.е. чтобы время жизни этого синглтона выходило за рамки экрана OrderActivity и сабкомпонента OrderComponent. Для этого надо создавать и хранить синглтон в другом компоненте, который живет подольше, чем наш OrderComponent. Можно использовать AppComponent.

Синглтон в AppComponent

Давайте компоненту AppComponent создадим scope аннотацию - AppScope

```
@Scope
@Retention(AnnotationRetention.RUNTIME)
annotation class AppScope
```

И добавим ее к компоненту

```
@AppScope
@Component(modules = [StorageModule::class, NetworkModule::class])
interface AppComponent
```

Теперь AppComponent готов к созданию синглтонов объектов. Для этого, как мы знаем, надо, чтобы объект был помечен scope аннотацией компонента - т.е. аннотацией AppScope.

Мы берем AppScope и добавляем ее вместо OrderScope к объекту OrderRepository:

```
@AppScope
class OrderRepository @Inject constructor()
```

Теперь мы могли бы создать у AppComponent метод getOrderRepository

```
@AppScope
@Component(modules = [StorageModule::class, NetworkModule::class])
interface AppComponent {
    fun getOrderRepository(): OrderRepository
    // ...
}
```

Этот метод создавал бы нам синглтон OrderRepository, время жизни которого равно времени жизни AppComponent.

Но мы так делать не будем. Потому что мы можем, как и раньше, получать OrderRepository из OrderComponent. Даггер дает нам такую возможность даже после смены scope аннотации у объекта OrderRepository.

Давайте рассмотрим подробно, что произойдет, если мы сейчас у OrderComponent вызовем метод getOrderRepository, чтобы получить репозиторий.

OrderComponent проверит scope аннотацию объекта OrderRepository. Он увидит, что scope аннотация у объекта есть - это AppScope. Но эта scope аннотация не совпадает со scope аннотацией сабкомпонента (OrderScope). Т.е. сабкомпонент, имея свое время жизни (OrderScope), должен создать синглтон объекта с другим временем жизни (AppScope). Сабкомпонент понимает, что ему это не под силу.

Если сабкомпонент сам не может создать объект, он просит этот объект у своего родительского компонента. Т.е. OrderComponent просит OrderRepository у AppComponent.

AppComponent создает OrderRepository и при этом видит, что scope аннотация объекта (т.е. AppScope) совпадает с его собственной scope аннотацией. В результате AppComponent создает синглтон OrderRepository. И передает этот синглтон сабкомпоненту OrderComponent.

В итоге мы от сабкомпонента OrderComponent получаем синглтон OrderRepository, который создается и хранится в AppComponent.

Посмотрим на исходники метода OrderComponent.getOrderRepository:

```
public OrderRepository getOrderRepository() {
      return DaggerAppComponent.this.orderRepositoryProvider.get();
}
```

Снова используется провайдер, который создает и хранит синглтон объекта OrderRepository. Но на этот раз провайдер берется из AppComponent.

Т.е. каждый новый экземпляр сабкомпонента OrderComponent не создает свой синглтон объекта OrderRepository, а берет его из компонента AppComponent. Это приводит к тому, что мы всегда получаем один и тот же OrderRepository при любом открытии экрана OrderRepository.

Давайте проверим. Откроем OrderActivity и вызовем метод getOrderRepository два раза подряд.

Результат:

```
orderRepository1 = ru.startandroid.daggercourse.order.OrderRepository@ff8e47d
orderRepository2 = ru.startandroid.daggercourse.order.OrderRepository@ff8e47d
```

Оба раза получаем один и тот же объект.

Теперь закроем OrderActivity, откроем его снова и вызовем getOrderRepository два раза. Результат:

```
orderRepository1 = ru.startandroid.daggercourse.order.OrderRepository@ff8e47d
orderRepository2 = ru.startandroid.daggercourse.order.OrderRepository@ff8e47d
```

Оба вызова вернули нам один и тот же объект. И этот тот же самый объект, что мы получали во время предыдущего открытия экрана. Хоть и сабкомпоненты были разные.

Таким образом мы расширили время жизни синглтона OrderRepository, просто поменяв у него scope аннотацию OrderScope на AppScope. Эта смена аннотации привела к тому, что создание объекта переместилось из OrderComponent в AppComponent. Но при этом мы продолжили получать этот объект из OrderComponent.

Теперь, кстати, синглтон OrderRepository похож на классический синглтон. Он живет все время работы приложения, потому что AppComponent живет все это время.

Имя аннотации

Давайте снова взглянем на варианты создания синглтона объекта OrderRepository:

Первый вариант:

```
@OrderScope
class OrderRepository @Inject constructor()
```

Второй вариант:

```
@AppScope
class OrderRepository @Inject constructor()
```

По имени аннотации мы сразу можем определить время жизни синглтона этого объекта. Именно поэтому у аннотации должно быть осмысленное имя, отражающее время жизни. А если аннотация называется просто Singleton - то не очень понятно время жизни такого объекта.

Provides

В рассмотренном нами примере объект OrderRepository имел аннотацию Inject в конструкторе. Компонент и сабкомпонент могли сами создавать его без всяких модулей.

А что если объект создается в модуле?

В этом случае просто добавляем scope аннотацию к Provides методу:

```
@Module
class OrderModule {

   @OrderScope
   @Provides
   fun provideOrderRepository(): OrderRepository {
       return OrderRepository()
   }

}
```

Этот модуль теперь может быть использован только в компоненте (или сабкомпоненте) с такой же scope аннотацией (OrderScope)

```
@OrderScope
@Subcomponent(modules = [OrderModule::class])
interface OrderComponent {

   fun getOrderRepository(): OrderRepository

}
```

Т.е. уже не получится просто так поменять у объекта scope аннотацию OrderScope на AppScope:

```
@Module
class OrderModule {

   @AppScope
   @Provides
   fun provideOrderRepository(): OrderRepository {
       return OrderRepository()
   }

}
```

Даггер будет ругаться, что мы сабкомпоненту OrderComponent с его аннотацией OrderScope подсовываем модуль с объектом с другой scope аннотацией (AppScope).

Поэтому нам придется этот Provides метод убрать из модуля OrderModule в какой-то другой модуль, который используется в AppComponent.

@Singleton

Обычно в уроках про scope используют аннотацию Singleton, которая доступна из коробки.

На мой взгляд, использовать ее для объяснения темы - не очень хорошая идея. Я ее использовал только в качестве примера неинформативного имени scope аннотации.

Ее, конечно, можно повесить на какой-то свой компонент и на объект, который хотите видеть синглтоном в этом компоненте. Но проблема в том, что глядя потом на этот объект, вы не понимаете, какое у него время жизни. Чтобы выяснить это, вам придется найти компонент, который использует эту scope аннотацию, и выяснить его время жизни.

Назад Вперёд

Комментарии

# **RE: Урок 11. Scope** — **Алексей** 21.04.2022 22:02
Отличное объяснение сути scope, спасибо!
Ответить

# **RE: Урок 11. Scope** — **Дмитрий** 28.08.2022 15:44
Урок - супер \ (￣▽￣) / , спасибо))
Ответить

# **AppScope** — **Евгений** 11.09.2022 17:22
Так и не понял, в примере создания синглтона с inject конструктором и Appscope сроком жизни нам в appcomponent не нужен метод getOrderRepository, хотя создавать синглтон должен appcomponent. Но в уроке про сабкомпоненты было сказано что компонент верхнего уровня ничего не знает о реализации создания объектов в сабкомпонентах. Вот в примере с модулем с appscope методом который лежит в appcomponent, тут все ясно.
Ответить

> \# **RE: AppScope** — **Dmitry Vinogradov** 20.09.2022 19:02
> "нам в appcomponent не нужен метод getOrderRepository" - это не значит, что appcomponent теперь не будет создавать этот объект. Будет. Потому что сабкомпонент попросит.
>
> Просто нам этот метод в appcomponent не нужен, т.к. мы этот объект получаем через OrderComponent
> Ответить

# **OrderModule** — **Даниил** 08.11.2023 10:53
Спасибо за урок, вы объясняете как всегда как бог. Одно замечание: в самом начале урока мы используем @Inject-аннотацию для OrderRepository, соответственно использовать модули нам не нужно. Но при этом чуть ниже написано:
@Subcomponent(modules = [OrderModule::class])
interface OrderComponent {
fun getOrderRepository(): OrderRepository
}
Вероятно, это копипаст) Просто вдруг кого-то введет в заблуждение
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.